Homework 3: Building Your Own Decentralized Exchange

Vaibhav Gera (24M0749)

1 Overview

In this two part assignment we implement a simple constant product Automated Market Maker (AMM) DEX and an arbitrage contract that performs arbitrage on two such DEXs. More specifically, the assignment includes :-

- **Token Contracts:** Deployment of two ERC-20 tokens (Token A and Token B).
- **DEX Contract:** Implementation of an Automated Market Maker (AMM) that supports liquidity provision, token swap, swap fee distribution and liquidity redemption.
- **Arbitrage Contract:** A contract that identifies and executes arbitrage opportunities between two deployed DEX contracts.
- **Simulation:** Two javascript based simulation scripts. One for simulating the DEX and gathering various metrics such as Total Value Locked, Reserve Ratio, Swap Volume, Slippage, etc. The other is for simulating arbitrage by showing two scenarios:- Successful and Unsuccessful arbitrage.

2 Implementation Details

2.1 Token Contract (Token.sol)

Both token A and token B are instantiation of the smart contract implemented in Token.sol. Token smart contract is implemented using the ERC-20 standard template available in Remix IDE. In addition to standard ERC-20 methods, it supports two additional functions mint() and burn() which can only be called by the owner of the token i.e. the person who called the constructor. This is done to simplify the distribution of tokens in the simulation script.

2.2 LP Tokens (LPToken.sol)

Extends ERC-20 standard template. Each Liquidity Provider Token (LPT) represents shares in the liquidity pool. LP tokens are minted proportionally when liquidity is added and burned when liquidity is withdrawn. Only the DEX contract is allowed to mint or burn these tokens.

2.3 DEX Contract (DEX.sol)

This is the core contract that implements the decentralized exchange functionalities.

2.3.1 Liquidity Management

- **Adding Liquidity:**
  - The first depositor sets the initial ratio between TokenA and TokenB.
  - Subsequent providers must deposit tokens preserving the reserve ratio, and receive LP tokens proportional to their contribution.
- **Withdrawing Liquidity:**
  - LP tokens are burned proportionally, and the corresponding amounts of TokenA and TokenB are received by the LP.
  - The withdrawal function computes the share based on current pool reserves.
- **Reserve Ratio & Spot Price:**
  - A function is provided to read the current reserves of both tokens.
  - The ratio of TokenA to TokenB serves as a dynamic spot price for trading.

2.3.2 Swap Functionality

- **Swap Mechanism:**
  - Users can swap TokenA for TokenB and vice versa.
  - The mechanism enforces the constant product invariant $x \times y = k$ (where x and y represent reserves).
  - A fee of 0.3% is applied on each swap. The fee is added to the reserves, benefiting liquidity providers.
- **Dynamic Pricing:**
  - The output amount for swaps is dynamically calculated based on reserve quantities and the fee, ensuring price adjustments as the reserve balance shifts.

2.4 Arbitrage Contract (arbitrage.sol)

2.4.1 Functionality

- **Price Comparison:** The contract retrieves spot prices from two DEX contracts and compares them. A profitable arbitrage opportunity exists when:

$$\frac{\text{Reserve2A}}{\text{Reserve1A}} \neq \frac{\text{Reserve2B}}{\text{Reserve1B}}$$

- **Execution:**
  - It calculates potential profit for arbitrage in both directions (A→B→A and B→A→B). For both DEX orders.
  - Swaps are executed only if the computed profit exceeds a pre-defined minimum threshold.
  - Upon a successful arbitrage, the contract returns the original capital along with the profit.

2.5 Testing and Simulation

2.5.1 Simulation Setup

- **JavaScript Simulation Scripts:**
  - **simulate_DEX.js:** Simulates addLiquidity(), removeLiquidity() and swap() functions on DEX.
  - **simulate_arbitrage.js:** Demonstrates two scenarios:
    * A profitable arbitrage execution.
    * A scenario where arbitrage is not executed due to insufficient profit.

3 Simulation Results:

3.1 DEX Simulation:

Metrics like:- Total Value Locked (TVL), Reserve Ratios, Swap Fees, Swap Volume, Liquidity Pool Token Distribution and Slippage are tracked on plotted with respect to the time in Fig. 1, Fig. 2 and Fig. 3.

3.2 Arbitrage Simulation:

The output for the arbitrage simulation is as follows:-

```
------------------------------------------------
Arbitrage Simulation Output:
------------------------------------------------
running scripts/simulate_arbitrage.js ...
Deployer:
0x5B38Da6a701c568545dCfcB03FcB875f56beddC4
Trader:
0xAb8483F64d9C6d1EcF9b849Ae677dD3315835cb2
```

Figure 1: DEX Simulation Metrics Overview

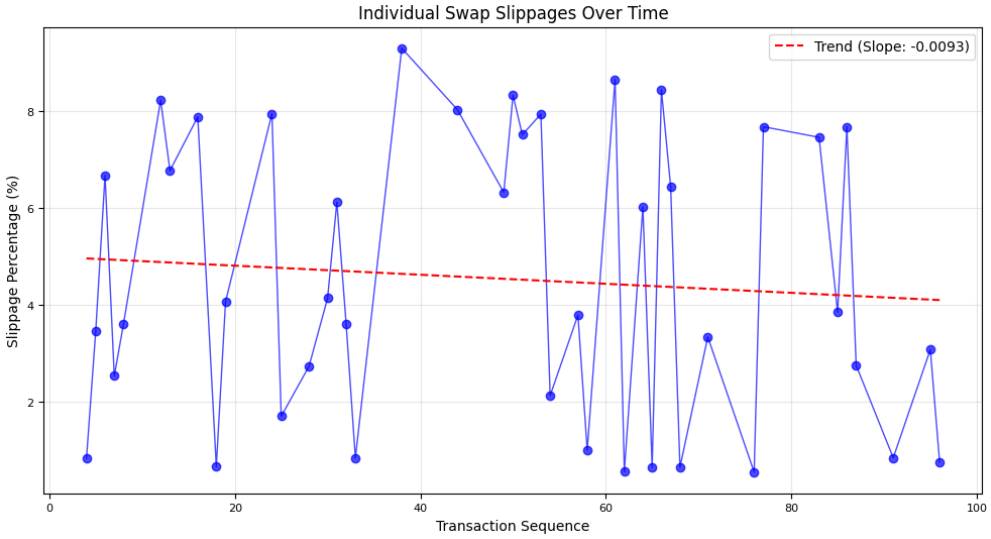

Figure 2: DEX Simulation: Slippage

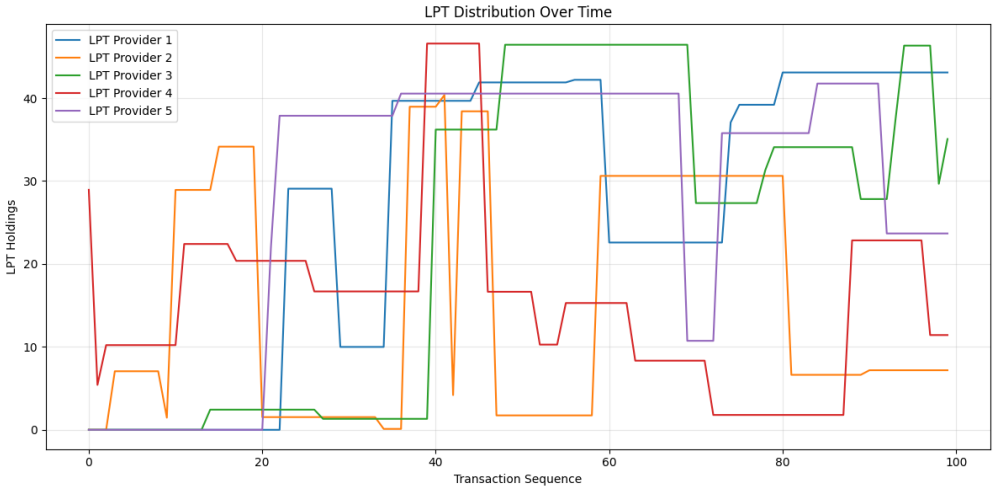

Figure 3: DEX Simulation: LPT Distribution

```
--- Scenario 1: Profitable Arbitrage Execution ---
Executing profitable arbitrage...
Trader Token A balance before arbitrage:
10
Arbitrage executed successfully
Trader Token A balance after arbitrage:
10.710211674838225

--- Scenario 2: Failed Arbitrage Execution (Insufficient Profit) ---
Executing insufficient profit arbitrage...
Trader Token A balance before arbitrage:
10.710211674838225
Arbitrage failed due to insufficient profit!
Transaction has been reverted by the EVM:

Simulation completed
------------------------------------------------
```

4 Theory Questions

Q1: Which address(es) should be allowed to mint/burn the LP tokens?
Answer: Only the DEX contract address should be allowed to mint and burn LP tokens.

Q2: In what way do DEXs level the playing ground between a powerful trader (HFT/institutional investor) and a lower resource trader (retail investor)?

Answer: DEXs are transparent the price is decided based on the ratio of the tokens which means all participants trade under the same fee structure and market conditions removing the advantage of exclusive information that HFT/institutional investor might have.

Q3: Suppose there are many transaction requests in a miner's mempool. How can the miner exploit this, and is it possible to mitigate such behavior?
Answer: Miners can front-run by reordering transactions to profit from information available in the mempool (Miner Extractable Value, MEV). Mitigation techniques include:

- Commit-Reveal schemes: Hiding trade information until execution.
- Fair Transaction Ordering: Making sure that if some transaction is received before another transaction then it is placed in the mined block first.
- Batch Processing: Executing transactions in groups to minimize reordering advantage.

Q4: How do gas fees influence the economic viability of the DEX and arbitrage?
Answer: Execution of transaction costs gas fee. Which means every action called on a smart contract will require gas fees proportional to the number of steps performed in the action. If the profit earned from the trade or an arbitrage opportunity is less than the gas fees to be paid for it then it can affect the viability of the DEX and arbitrage.

Q5: Could gas fees lead to undue advantages for some transactors? How?
Answer: Yes, traders with more money can afford to pay higher gas fee giving them a priority in execution of the transaction.

Q6: What are the various ways to minimize slippage in a swap?
Answer: Slippage can be minimized by:

- Increasing liquidity the size of liquidity pool.
- Limiting trade sizes relative to the total liquidity.

Q7: How does slippage vary with the "trade lot fraction" in a constant product AMM?
Answer: For a constant product AMM:

- When the trade lot fraction is small, the slippage is small because the impact on the reserve is not as musch
- As the trade lot fraction increases, there's an increase in slippage as the impact on reserves is more

5 Conclusion

This project demonstrates the development of a basic decentralized exchange using Solidity. It covers key components of DeFi protocols: token deployment, automated market making, liquidity management, fee-based swap execution, and arbitrage detection/execution.